

ECUE MIAGE : Application Web

Rapport de conception



Hamadouche Zakary

Ozdoev Akramat

Table des matières.

1. Introduction	2
2. Descriptif du dépôt	3
3. Architecture logicielle et choix de conception	3
3.1 Modélisation Commune et Modularité.....	4
3.2 Météorite Canvas.	4
3.3 GamesOnWeb Foot 3D.	4
3.4 Neon ZIP.	4
4. L'utilisation de l'IA dans le projet	5
5. Conclusion	5

1. Introduction

La mission principale de ce projet a consisté à développer un site web unifié regroupant trois jeux. Cette plateforme propose un portail commun qui intègre un système de connexion pour l'authentification des joueurs ainsi qu'un dispositif de sauvegarde des scores pour chacun des trois jeux développés.

Le premier jeu intitulé Météorite Canvas repose sur l'utilisation de la technologie Canvas HTML5 pour la création d'un jeu d'arcade spatial. Le second jeu a été conçu spécifiquement pour le concours GamesOnWeb et propose une simulation de football développée avec le moteur Babylon.js. Enfin le troisième projet nommé Neon ZIP se focalise sur la manipulation avancée du DOM pour offrir un jeu de réflexion logique basé sur des grilles interactives.

2. Descriptif du dépôt

L'architecture matérielle et réseau de notre application repose sur une séparation claire entre le client et l'API ainsi que la base de données.

Le Frontend est entièrement hébergé sur la plateforme Vercel. Il contient le Hub d'accueil ainsi que les répertoires dédiés aux trois jeux respectifs. Le Backend est quant à lui hébergé sur Railway avec un serveur Node.js exploitant le framework Express pour agir comme une API REST. Cette API gère les requêtes d'authentification par jetons ainsi que les transactions liées aux profils des joueurs.

La base de données MongoDB Atlas permet quant à elle le stockage persistant des collections d'utilisateurs ainsi que des scores détaillés et des historiques de matchs. Nous avons mis en place un système de configuration dynamique via un script de génération de fichier de configuration pour assurer la liaison entre le domaine Vercel et le serveur Railway.

L'organisation des fichiers sources est rigoureusement standardisée pour les trois projets afin de faciliter la lecture du code. Le dossier assets centralise les médias comme les fichiers sonores au format MP3 ou les modèles 3D au format GLB. Le dossier css regroupe les styles visuels avec une forte utilisation des variables pour maintenir une identité graphique commune. Le dossier js contient le cœur logique ainsi que les définitions de classes pour les entités de jeu tandis que le dossier ui s'occupe exclusivement de la couche de présentation et de l'interface graphique.

3. Architecture logicielle et choix de conception

3.1 Modélisation Commune et Modularité.

Nous avons structuré l'intégralité de nos codes en modules ES6 pour permettre un couplage faible entre les différents composants logiciels. Le flux de données suit globalement un pattern MVC simplifié où l'état du jeu est isolé dans des objets de state puis mis à jour par des gestionnaires de logique dédiés.

3.2 Météorite Canvas.

Pour ce projet le défi majeur résidait dans la gestion d'un grand nombre d'objets interactifs et de projectiles sur le plan 2D. Nous avons opté pour une structure d'héritage profonde avec une classe parente appelée *ObjetGraphique* qui gère les propriétés fondamentales comme la position ou la vitesse ainsi que la méthode de rendu de base.

Les classes spécialisées comme *Vaisseau* ou *Meteorite* étendent cette base pour y ajouter des comportements propres tels que le tir ou l'explosion. Nous avons également implémenté un *ParticleManager* sophistiqué pour gérer les effets visuels de débris. Les collisions sont calculées via des algorithmes de détection de cercles et de rectangles.

3.3 GamesOnWeb Foot 3D.

Ce projet constitue le sommet technique de notre TP en raison de l'utilisation de la 3D et d'IA complexes. La caméra broadcast est un élément dont nous sommes particulièrement fiers car elle est divisée en trois couches logicielles distinctes. Le *Brain* analyse l'action en temps réel tandis que le *Rig* calcule les transformations de vue pour que le *Runtime* puisse appliquer une interpolation douce garantissant un rendu digne d'une retransmission télévisuelle professionnelle.

L'intelligence artificielle des joueurs repose sur le paradigme des *Steering Behaviors* qui permet aux joueurs autonomes de prendre des décisions de placement en fonction de forces vectorielles. Un attaquant sera attiré par le but adverse tandis qu'un défenseur cherchera à intercepter la trajectoire du ballon. Nous avons également intégré un support complet pour les manettes de jeu via l'API *Gamepad*.

3.4 Neon ZIP.

La conception de ce puzzle game repose sur une délégation asynchrone des calculs les plus lourds. La recherche d'un chemin hamiltonien complet dans une grille est un

problème complexe qui peut ralentir l'interface utilisateur. Pour résoudre cela nous avons isolé toute la logique de génération dans un Web Worker qui communique avec le thread principal par messages.

L'architecture des obstacles utilise le polymorphisme pour simplifier le calcul du tracé. Qu'il s'agisse d'une case bloquée ou d'un côté interdit le moteur de jeu interroge simplement une méthode universelle de blocage. Cela nous a permis d'ajouter de nouveaux types de difficultés sans modifier le code source du moteur de tracé.

4. L'utilisation de l'IA dans le projet

L'intelligence artificielle a été utilisée comme un collaborateur technique pour nous aider à structurer nos idées et à résoudre des points de blocage spécifiques. Elle a servi d'architecte logiciel consultatif pour valider des structures de données complexes ou pour nous suggérer des optimisations algorithmiques.

Pour le jeu de football l'assistant nous a aidés à concevoir la logique des comportements de direction que nous avons ensuite codée manuellement pour l'adapter aux règles du sport. Pour Neon ZIP l'IA nous a orientés vers l'heuristique de Warnsdorff qui permet de trouver des solutions de chemins beaucoup plus rapidement dans des grilles larges. En ce qui concerne l'aspect visuel l'IA a suggéré des orientations esthétiques comme le style glassmorphism que nous avons traduit en code CSS pur pour nos menus et nos interfaces de scores.

5. Conclusion

Ce projet nous a permis de comprendre l'importance d'une infrastructure backend pour soutenir des applications frontales. Les perspectives d'évolution pour ces jeux sont nombreuses notamment avec l'ajout possible d'un mode multijoueur en ligne via des websockets ou encore l'amélioration des graphismes grâce à des shaders personnalisés. Nous avons réussi à créer une plateforme cohérente et engageante qui répond aux exigences du tp tout en reflétant notre passion pour le développement de jeux vidéo.